



University of
Inland Norway

ØKA2015 Applied Data Science

Autumn 2026 | 7.5 study points | Lillehammer

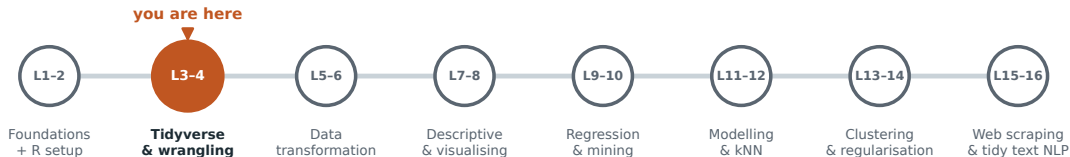
Lecture 4

Data Lab: importing and transforming data with dplyr

Touseef Hameed | `touseef.hameed@inn.no`

Friday 18 September 2026 | 10:15–12:00

Where we are in the course



Last time (Lecture 3, Wednesday)

Objects, vectors and functions. The tidyverse installed. A CSV file read into R with `read_csv()` and inspected with `glimpse()`.

Today, a Data Lab

You start changing data: keep the rows you want, put them in order, count what is there. Four verbs and the pipe, typed along as we go.

Materials for this lecture

The work is done in **RStudio on your own laptop**, typing each command yourself. Everything below is also online.

Slides | this document, as a PDF

R script | `lecture04_wrangling.R`: every command from today, in today's order, each line explained in a comment. We run it in parallel.

R Markdown tutorial | `lecture04_wrangling.Rmd` and its knitted PDF: the same content as a document

Practice exercise | short tasks on the flights data, not graded

Data | the `flights` table, inside the `nycflights13` package

Primary: RStudio on your laptop, where Assignment 1 is written. GitHub and Colab are for afterwards, to download or run the code.

Where it is

GitHub, the course repository:

[github.com/touseefhameed/
oka2015-applied-data-science](https://github.com/touseefhameed/oka2015-applied-data-science)



Google Colab, the script in your browser, nothing to install: [Open in Colab](#), also a badge on the GitHub page

Reading

Wickham et al., *R for Data Science* (2e), Ch. 3 *Data transformation*, up to the end of *Rows*.
Free online: <https://r4ds.hadley.nz>

Today's plan

- 1 **Why transform data**, and what dplyr is
- 2 **The data**: `nycflights13`, how to see all of it
- 3 **dplyr basics**: what verbs share, the pipe, `na.rm`
- 4 **Rows**: `filter()`, the operators, two common mistakes
- 5 **Rows**: `arrange()`, `distinct()`, `count()`
- 6 **Check**, then the project file and R Markdown

How the lab runs

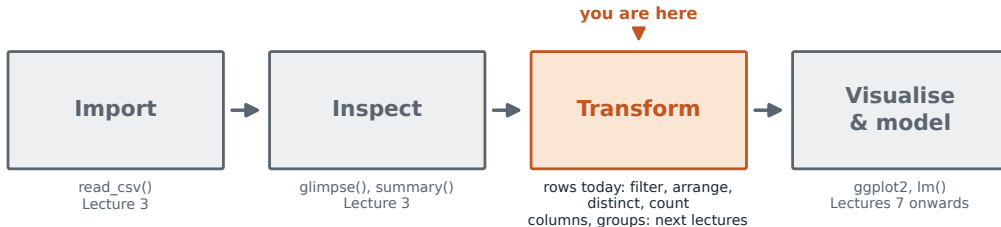
Each code slide is one block of the script. I type it, you type it, we read the output together. The first line of every result says how many rows came back. That is how you check a verb.

By the end you will be able to

- ✓ load a dataset that ships inside a package
- ✓ write a condition with the comparison and logical operators
- ✓ keep the rows that meet it
- ✓ sort a table, and count what is in a column
- ✓ chain steps with the pipe and read them in words

Columns and groups: the next lectures.

1. Why transform data



Data transformation | changing a table into the table you need: the right rows, in the right order, with the right columns and summaries. Also called *data wrangling*.

Why | data rarely arrives in the form a question needs. In Lecture 2 we said this step takes **50 to 80%** of a project's time.

R for Data Science (2e), Ch. 3 Introduction, <https://r4ds.hadley.nz/data-transform>

Key idea

Wednesday you read a table and looked at it. Today you change it. Every change is one short *verb*, and verbs chain with the pipe.

1. dplyr: a grammar of data manipulation

ROWS	COLUMNS	GROUPS
<i>which observations: today</i>	<i>which variables: next lectures</i>	<i>per category: next lectures</i>
<code>filter()</code> rows by condition	<code>select()</code> columns by name	<code>group_by()</code> split into groups
<code>arrange()</code> sort rows	<code>mutate()</code> new columns from old	<code>summarise()</code> one row per group
<code>distinct()</code> unique rows	<code>rename()</code> change a name	<code>n()</code> count in a group
<code>count()</code> how many of each	<code>relocate()</code> move columns	<code>slice_max()</code> top rows per group

joined into one sentence by the pipe `|>` read as “then”

dplyr is a set of **verbs**. Each verb does one thing to a table and is named after what it does. The verbs are grouped by what they act on: rows, columns, groups. Today: the four verbs for rows.

R for Data Science (2e), Ch. 3 *dplyr* basics; <https://dplyr.tidyverse.org>



one of the nine
tidyverse packages,
loaded by
`library(tidyverse)`

2. Prerequisites: two packages

```
install.packages("tidyverse") # once per
  computer
install.packages("nycflights13") # once per
  computer
library(tidyverse) # every session
library(nycflights13)
```

```
-- Attaching core tidyverse packages ----
v dplyr 1.2.0 v readr 2.1.5
v forcats 1.0.0 v stringr 1.5.1
...
-- Conflicts -----
x dplyr::filter() masks stats::filter()
x dplyr::lag() masks stats::lag()
```

The two install lines are at the top of the script. Run them once. Running them again does no harm; it only takes time.

R for Data Science (2e), Ch. 3 Prerequisites

nycflights13 | a package whose content is *data*: every flight that left New York in 2013. A fourth way to get data into R, after the three of Wednesday.

The Conflicts note | two packages have a function called **filter**. The tidyverse one is used. This is information, not an error.

package::function() | says which one you mean. **dplyr::filter()** keeps rows; **stats::filter()** is for time series. **nycflights13::flights** reads “flights from nycflights13”.

2. The data: flights, and how to see all of it

```
flights <- nycflights13::flights
flights
```

```
# A tibble: 336,776 x 19
  year month day dep_time sched_dep_time
<int> <int> <int> <int> <int>
1 2013 1 1 517 515
2 2013 1 1 533 529
3 2013 1 1 542 540
# i 336,773 more rows, 14 more variables
```

```
View(flights) # spreadsheet viewer
print(flights, width = Inf) # every column
print(flights, n = Inf) # every row
glimpse(flights) # one line per column
```

R for Data Science (2e), Ch. 3 nycflights13; ?flights for the meaning of every column

A tibble | the tidyverse's data frame. It prints the first rows and only the columns that fit, and says what it left out.

336,776 rows | one per flight. 19 columns. Too large to check by eye, so you need the verbs.

Why this dataset | it is the dataset used in the textbook, and the next lectures continue with it.

2. Reading `glimpse()`: names, types, first values

```
Rows: 336,776
Columns: 19
$ month <int> 1, 1, 1, 1, 1, ...
$ dep_time <int> 517, 533, 542, ...
$ dep_delay <dbl> 2, 4, 2, -1, -6, ...
$ arr_delay <dbl> 11, 20, 33, -18, ...
$ carrier <chr> "UA", "UA", "AA", ...
$ dest <chr> "IAH", "IAH", "MIA", ...
$ distance <dbl> 1400, 1416, 1089, ...
```

```
head(flights$dep_delay)
mean(flights$distance)
```

```
[1] 2 4 2 -1 -6 -4
[1] 1039.913
```

Types | `<int>` whole numbers, `<dbl>` decimals, `<chr>` text, `<dtm>` date-time. The type decides what you can do: arithmetic on numbers, comparisons on all.

Units | delays in minutes, negative means early. `dep_time` 517 is 05:17. `distance` in miles, `air_time` in minutes.

NA in `dep_delay` | the flight was cancelled.

\$ | pulls one column out as a plain vector. A vector goes straight into `head()`, `mean()`, `sum()`, `max()`.

3. What every dplyr verb shares

Three things

- 1 The **first argument is a data frame**.
- 2 The next arguments say **which columns**, by bare name, no quotes.
- 3 The output is a **new data frame**. The input is never changed.

```
filter(flights, month == 1)
```

```
# A tibble: 27,004 x 19
```

The verb, then the table, then what to do with it: the January flights.

R for Data Science (2e), Ch. 3 dplyr basics

Why 1 matters | the table can drop into the first slot of the next verb. That is what the pipe does, next slide.

Why 2 matters | `month`, not `"month"`. A bare name is a column. Text *values* keep their quotes: `dest == "IAH"`.

Why 3 matters | you can try anything. `flights` stays as it is until you save a result with `<-`.

Four groups of verbs | rows, columns, groups, tables. Each verb does one thing, so a real question combines several.

3. The pipe `|>`: read it as “then”

`x |> f(y)` means `f(x, y)`

the thing on the left becomes
the first argument on the right

`16 |> sqrt()`

`sqrt(16)`

both give 4

read `|>` as “then”

```
sqrt(16)
16 |> sqrt()
flights |> nrow()
flights |> filter(month == 1)
```

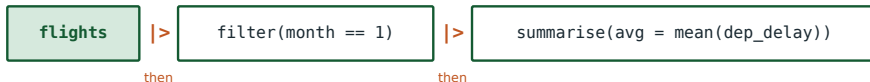
```
[1] 4
[1] 4
[1] 336776
# A tibble: 27,004 x 19
```

`x |> f(y)` is `f(x, y)`. The thing on the left becomes the first argument on the right. By rule 1, that is the table.

Keyboard | **Ctrl + Shift + M**. If it inserts `%>%`: Tools, Global Options, Code, tick *Use native pipe operator*.

`%>%` | the older pipe. In this course the two are the same. The book and these slides use `|>`.

3. The pipe `|>`: read it as “then”



take the flights,

then keep January,

then average the departure delay.

each verb receives what the previous one produced

```
sqrt(16)
16 |> sqrt()
flights |> nrow()
flights |> filter(month == 1)
```

`x |> f(y)` | is `f(x, y)`. The thing on the left becomes the first argument on the right. By rule 1, that is the table.

Keyboard | **Ctrl + Shift + M**. If it inserts `%>%`: Tools, Global Options, Code, tick *Use native pipe operator*.

`%>%` | the older pipe. In this course the two are the same. The book and these slides use `|>`.

3. A first pipeline, and `na.rm = TRUE`

```
flights |>
  filter(month == 1) |>
  summarise(avg_delay = mean(dep_delay))
```

```
avg_delay
1 NA
```

```
v <- c(1, 2, NA, 4)
mean(v) # NA
mean(v, na.rm = TRUE) # 2.333333
flights |>
  filter(month == 1) |>
  summarise(avg_delay =
    mean(dep_delay, na.rm = TRUE))
```

```
avg_delay
1 10.04
```

In words | take the flights, then keep January, then average the departure delay. `summarise()` turns many values into one row.

= and <- | inside a function call, `=` gives a result a *name*: `avg_delay`. Outside, `<-` stores an *object*: `flights <- ...`. Not interchangeable.

Why NA | 8,255 cancelled flights have no delay. The mean of values that include a missing one is missing.

na.rm = TRUE | “remove NA” for this one calculation. It goes inside `mean()`, `sum()`, `max()`. January flights left 10 minutes late on average. The data itself is unchanged.

4. Rows

`filter()`, `arrange()`, `distinct()`, `count()`

Which observations, and in what order

4. filter() keeps the rows that meet a condition

`filter(dep_delay > 120)`

month	dep_delay	dest	distance	air_time
1	2	IAH	1400	227
1	853	BHM	866	148
3	-4	ORD	719	150
6	144	MIA	1089	160
12	-6	ATL	762	116

flights (five rows, simplified)



month	dep_delay	dest	distance	air_time
1	853	BHM	866	148
6	144	MIA	1089	160

result: a new tibble

rows that meet the condition are kept; the columns are untouched

```
flights |> filter(dep_delay > 120)
```

```
# A tibble: 9,723 x 19
  year month day dep_time dep_delay
<int> <int> <int> <int> <dbl>
1  2013   1   1  848    853
2  2013   1   1  957    144
```

Definition | `filter()` keeps the rows for which a condition is TRUE. The columns are untouched.

In words | flights, then keep the rows whose departure delay is over 120 minutes: 9,723 of 336,776.

The condition | `dep_delay > 120` is TRUE or FALSE for every row. Conditions are built from the operators on the next slide.

4. The operators a condition is built from

Comparison: one value against another, gives TRUE or FALSE

operator	meaning	example
<code>==</code>	equal to	<code>dest == "IAH"</code>
<code>!=</code>	not equal to	<code>origin != "JFK"</code>
<code>></code>	greater than	<code>dep_delay > 120</code>
<code>>=</code>	greater or equal	<code>arr_delay >= 120</code>
<code><</code>	less than	<code>distance < 500</code>
<code><=</code>	less or equal	<code>dep_delay <= 0</code>
<code>%in%</code>	is one of	<code>month %in% c(7, 8, 9)</code>

Logical: combine conditions

operator	meaning	example
<code>&</code>	AND, both hold	<code>month == 1 & day == 1</code>
<code> </code>	OR, at least one	<code>month == 1 month == 2</code>
<code>!</code>	NOT	<code>!is.na(dep_delay)</code>

Two equals signs

`==` asks "is it equal?". A single `=` sets something. Text is compared exactly: `"JFK" == "jfk"` is FALSE.

R for Data Science (2e), Ch. 3 `filter()`. Arithmetic (+ - * / ^) was Lecture 3.

4. filter() with several conditions

```
flights |> filter(month == 1 & day == 1)
```

```
# A tibble: 842 x 19
```

```
flights |> filter(month == 1 | month == 2)
flights |> filter(month %in% c(1, 2))
```

```
# A tibble: 51,955 x 19
# A tibble: 51,955 x 19
```

```
flights |> filter(dest == "IAH")
flights |> filter(!is.na(dep_delay))
```

```
# A tibble: 7,198 x 19
# A tibble: 328,521 x 19
```

& | AND. January *and* the first: New Year's Day, 842 flights.

| and %in% | OR. January *or* February, two ways to write it. **%in%** is shorter, and **month %in% 7:9** is summer.

Text | values in quotes, "IAH". The column name is not.

is.na() | TRUE where the value is missing. With **!** in front: the flights that actually departed.

Your turn: flights to Houston in July. Which two conditions, and joined by what?

4. Two common mistakes

Mistake 1: one equals sign

```
flights |> filter(month = 1)
```

```
Error in 'filter()':  
! We detected a named input.  
i This usually means that you've used  
  '=' instead of '=='.  
i Did you mean 'month == 1'?
```

R stops and says what it thinks you meant. Read the message.

R for Data Science (2e), Ch. 3 Common mistakes

Mistake 2: “or” written as in English

```
flights |> filter(month == 1 | 2)
```

```
# A tibble: 336,776 x 19
```

No error, and every row comes back. R reads it as `(month == 1) | 2`. The number 2 counts as TRUE, so the condition is always TRUE. **Wrong, and no warning.**

Write instead

```
month == 1 | month == 2    or    month %in%  
c(1, 2)
```

4. A verb never changes its input: use `<-` to keep a result

```
flights |> filter(month == 1 & day == 1)
nrow(flights)
```

```
# A tibble: 842 x 19
[1] 336776
```

The 842 rows were printed and forgotten. `flights` still has all 336,776.

```
jan1 <- flights |> filter(month == 1 & day
  == 1)
nrow(jan1)
```

```
[1] 842
```

R for Data Science (2e), Ch. 3 `filter()`, last paragraph

Key idea

dplyr functions never modify their inputs. A verb returns a new table. Nothing is stored until you give the result a name with `<-`. Then a second object appears in the Environment pane, and the original is untouched.

Working method | run the pipeline without `<-` first and look at the output. When it is right, add the name.

Naming | `jan1`, `late_flights`, `houston`: say what is inside. Lower case, underscores, as on Wednesday.

Your turn 1: filter()

Find all flights that

- 1 had an arrival delay of two or more hours
- 2 flew to Houston: "IAH" or "HOU"
- 3 were operated by United, American or Delta: "UA", "AA", "DL"
- 4 departed in summer: July, August, September
- 5 arrived more than two hours late but did not leave late

For each, read the row count in the first line of the output.

Your turn 1: filter()

Find all flights that

- ① had an arrival delay of two or more hours
- ② flew to Houston: "IAH" or "HOU"
- ③ were operated by United, American or Delta: "UA", "AA", "DL"
- ④ departed in summer: July, August, September
- ⑤ arrived more than two hours late but did not leave late

For each, read the row count in the first line of the output.

R for Data Science (2e), Ch. 3 Rows, Exercise 1

Answers: the condition inside filter()

```
arr_delay >= 120 # 10200
dest %in% c("IAH", "HOU") # 9313
carrier %in% c("UA", "AA", "DL") # 139504
month %in% 7:9 # 86326
arr_delay > 120 & dep_delay <= 0 # 29
```

Note 2 and 3: “or” between values of one column is `%in%`, never `&`.

4. arrange() changes the order of the rows

`arrange(desc(dep_delay))`

month	dep_delay	dest	distance	air_time
1	2	IAH	1400	227
1	853	BHM	866	148
3	-4	ORD	719	150
6	144	MIA	1089	160
12	-6	ATL	762	116

→

month	dep_delay	dest	distance	air_time
1	853	BHM	866	148
6	144	MIA	1089	160
1	2	IAH	1400	227
3	-4	ORD	719	150
12	-6	ATL	762	116

same rows, same columns, new order: largest delay first

```
flights |> arrange(desc(dep_delay))
```

```
# A tibble: 336,776 x 19
  year month day dep_time dep_delay
1 2013 1 9 641 1301
```

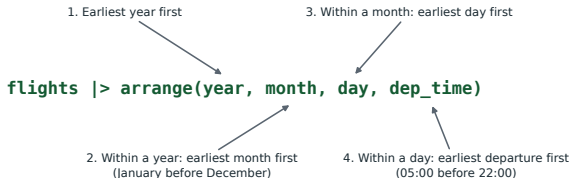
Definition | `arrange()` sorts the rows by the columns you name. Ascending by default; `desc()` for descending. It never removes a row.

The record | 1,301 minutes, 21.7 hours, on 9 January.

Next slide | several columns at once, and the `desc()` syntax.

R for Data Science (2e), Ch. 3 arrange()

4. arrange() with several columns, and desc()



```
flights |> arrange(year, month, day, dep_time)
flights |> arrange(desc(dep_time))
```

```
year month day dep_time dest
1 2013 1 1 517 IAH # first call
1 2013 10 30 2400 BQN # second call
```

Several columns | the first column decides the order. Each later column only breaks ties in the ones before it. Result: chronological order.

desc() syntax | wraps exactly one column. `desc(dep_time)`: latest departure first. Two columns: `desc(month)`, `desc(day)`.

4. `distinct()` and `count()`: which values occur, and how often

```
flights |> distinct(origin, dest)
```

```
# A tibble: 224 x 2  
  origin dest  
1 EWR IAH  
2 LGA IAH
```

```
flights |> count(origin, dest, sort = TRUE)
```

```
# A tibble: 224 x 3  
  origin dest n  
1 JFK LAX 11262  
2 LGA ATL 10263
```

```
flights |>  
  distinct(year, month, day) |> nrow()
```

```
[1] 365
```

`distinct()` | each unique combination of the named columns, once. New York flew 224 routes in 2013.

`count()` | the same combinations with how many rows each, in a column `n`. `sort = TRUE`: largest first. JFK to Los Angeles, 11,262 flights. The first question to ask of any text column.

Every day | 365 distinct dates: a flight on every day of 2013.

Your turn 2: `arrange()`, `distinct()`, `count()`

- 1 Sort to find the flights that left earliest in the morning. What is `dep_time` 1?
- 2 Which flights travelled the farthest? Which the least?
- 3 How many flights did each airline operate, largest first?
- 4 Which destination had the most flights?
- 5 Does it matter whether you `filter()` first and then `arrange()`, or the reverse?

Your turn 2: `arrange()`, `distinct()`, `count()`

- 1 Sort to find the flights that left earliest in the morning. What is `dep_time` 1?
- 2 Which flights travelled the farthest? Which the least?
- 3 How many flights did each airline operate, largest first?
- 4 Which destination had the most flights?
- 5 Does it matter whether you `filter()` first and then `arrange()`, or the reverse?

Answers

```
arrange(dep_time) # 1 = 00:01
arrange(desc(distance)) |> glimpse()
arrange(distance) |> glimpse()
count(carrier, sort = TRUE) # UA 58665
count(dest, sort = TRUE) # ORD 17283
```

each after `flights |>`. 2: JFK to Honolulu, 4,983 miles; Newark to LaGuardia, 17 miles. `dest` and `distance` are off-screen in the normal printout; `glimpse()` shows every column of the top rows. 5: same result either way; filtering first sorts fewer rows, so it is faster.

Check your understanding

1. How do you keep the result of this as an object called `jan1`, and then look at it?

```
flights |> filter(month == 1 & day == 1)
```

2. What is the error here?

```
flights |> filter(month = 1)
```

3. This runs without error. Does it do what was meant?

```
flights |> filter(month == 1 | 2)
```

4. After `flights |> arrange(desc(distance))`, how many rows does `flights` have?

Check your understanding

1. How do you keep the result of this as an object called `jan1`, and then look at it?

```
flights |> filter(month == 1 & day == 1)
```

2. What is the error here?

```
flights |> filter(month = 1)
```

3. This runs without error. Does it do what was meant?

```
flights |> filter(month == 1 | 2)
```

4. After `flights |> arrange(desc(distance))`, how many rows does `flights` have?

Answers

1. `jan1 <- flights |> filter(month == 1 & day == 1)`, then `jan1` or `View(jan1)`.

2. One equals sign assigns. The comparison is `==`. R says so in the message.

3. No. The 2 counts as TRUE, so every row passes. Write `month == 1 | month == 2` or `month %in% c(1, 2)`.

4. 336,776. The sorted table was printed, not stored. `flights` is unchanged either way.

The project file and the R Markdown file

The project file, .Rproj

File, New Project, Existing Directory, then pick your folder. Or double-click a `.Rproj` file you were given.

RStudio then opens in that folder, so `read_csv("data/housing.csv")` finds the file without `setwd()`. One folder, one project.

The lab folders

`lecture03.Rproj`, `lecture04.Rproj` and `assignment1.Rproj` each open their own folder. Start from the project file, not from a script inside it.

The R Markdown file, .Rmd

File, New File, R Markdown. Three parts: the header between the `---` lines, text, and code chunks between ````{r}` and `````.

Knit runs every chunk in a clean session and writes a PDF or HTML next to the file: code, output and explanation together.

For Assignment 1

Work in the script first. When a command is right, copy it into the chunk under the matching heading of the template, write one sentence, knit. `rmarkdown_guide.pdf` walks through it.

Course materials online

Everything from this lecture is online, and stays there for the whole course:

GitHub | github.com/touseefhameed/oka2015-applied-data-science

the slides, the R script, the R Markdown document and its PDF, and the practice exercise, in the `lecture04` folder. The assignment folder is beside it.

Google Colab | [Open in Colab](#), or the second badge on the GitHub page
the same script, running in your browser. No installation.

Colab in three steps

Open the link → sign in with any Google account → press the play button on the first cell. It installs `nycflights13` for you, about a minute.

Primary and secondary

Primary: RStudio on your own laptop, typing the commands yourself. That is where Assignment 1 is written.

Afterwards: download the script from GitHub to rerun it, or run it directly in Colab with nothing installed.

Scan with your phone



GitHub



Colab, Lecture 4

What we covered today

Concepts

- ✓ a package can ship data; `package::object`
- ✓ a tibble prints what fits; `View`, `width = Inf`, `glimpse`
- ✓ three things every verb shares
- ✓ the pipe `|>`, read as “then”
- ✓ `==` compares, `=` does not; “or” is `|` or `%in%`
- ✓ a verb never changes its input; `<-` keeps a result

Tools

- ✓ `filter()` with `==` `!=` `>` `>=` `<` `<=` `&` `|` `!` `%in%` `is.na()`
- ✓ `arrange()` and `desc()`, one key or several
- ✓ `distinct()`, `count(sort = TRUE)`
- ✓ `summarise()` with `na.rm = TRUE`, a first look
- ✓ the project file and the R Markdown file

If you can write a filter with two conditions and say what it returns before running it, today's objective is met.

Assignment 1, and before the next lecture

Assignment 1: readr and dplyr

On Canvas since Wednesday. Due **Wednesday 30 September, 23:59**.

Eight problems. Six require short pieces of R code, two require short written answers. You submit a script, and should document the work in R Markdown; screenshots of your RStudio window are accepted if knitting fails.

Start this weekend

Wednesday's lecture and today's cover most of it. **Problems 5 and 6** need the two column verbs, which come in Lecture 5 on **Tuesday**, a week before the deadline. Write the script first, then move the working commands into the R Markdown template.

Today's verbs on the assignment data

It is a CSV, read as on Wednesday. Which districts are **INLAND**? Which are worth over 500,000? How many of each **ocean_proximity**? Good practice with **filter()** and **count()**.

Before Tuesday

- ✓ finish the script if you did not
- ✓ the practice exercise
- ✓ read *R for Data Science* Ch. 3 up to *Columns*

Next lectures

Lecture 5, Tuesday 22 September

dplyr continues: the column verbs.

Lecture 6, Thursday 24 September

dplyr continues: groups and data transformation.

Both use the same `flights` table. Bring the laptop with `nycflights13` installed.

The thread

Wednesday: data *into* R.

Today: which *rows*, in what order.

Next: which *columns*, and one answer per *group*.

Sources used in this lecture

Course reading

- Wickham, H., Çetinkaya-Rundel, M. & Grolemund, G. (2023). *R for Data Science* (2nd ed.). O'Reilly. Ch. 3 *Data transformation*: Introduction, Prerequisites, nycflights13, dplyr basics, Rows. Free online: <https://r4ds.hadley.nz/data-transform>
- Wright, C., Ellis, S., Hicks, S. & Peng, R. (2021). *Tidyverse Skills for Data Science in R*, Ch. 3 *Data Wrangling*. <https://jhudatascience.org/tidyversecourse/>

Packages and data

- dplyr: <https://dplyr.tidyverse.org> cheat sheet: <https://posit.co/resources/cheatsheets/>
- nycflights13: Wickham, H. (2021). Flights that departed NYC in 2013. US Bureau of Transportation Statistics. <https://github.com/tidyverse/nycflights13>

Course material

- GitHub: <https://github.com/touseefhameed/oka2015-applied-data-science> Google Colab: [Open in Colab](#)

Thank you



Course materials: github.com/touseefhameed/oka2015-applied-data-science

Touseef Hameed | touseef.hameed@inn.no